VitalGuard AI v4.3.7

Security Audit Partner Manual

Remediation Summary, Architectural Strengths, and Honest Disclosure of Residual Risks

Prepared by Morgan J. (Gyu-min Jeon) — M-Corp Ethical AI

Document Control

Document Title	VitalGuard v4.3.7 — Security Audit Partner Manual
Document Purpose	All corrective actions applied since the initial deliverables were submitted to the independent security evaluator, along with the current security status.
Prepared by	Morgan J. (Gyu-min Jeon), M-Corp Ethical AI
Contact	contact@mcorpai.org • mcorpai.org
Date of Issue	2026
Reference	OTF Security Lab Application #21441
Confidentiality	Shared under responsible-disclosure principles
License Context	M-Corp Ethical AI License (Hippocratic 3.0 Derivative). Exclusively for civilian, agricultural, and humanitarian use.

1. Executive Summary

VitalGuard is a single-file, offline-first humanitarian AI application intended for deployment in low-connectivity, high-risk environments where conventional internet-dependent solutions are not viable. Because the software's users include refugee households, humanitarian field workers, and activists, the correctness of its security claims carries a direct human cost. This manual exists so that any external security assessor can, within a short reading time, understand exactly what has been changed, what has been deliberately left as-is, and where independent scrutiny would be most valuable.

The architectural strengths that the earlier audit identified (Content-Security-Policy with connect-src 'none', a Permissions-Policy that disables camera / payment / USB, PBKDF2-based AES-GCM encrypted backup, a custom Sanitizer module, and the complete absence of fetch / XMLHttpRequest / WebSocket calls) have been preserved intact. What v4.3.6 and v4.3.7 add is the removal of the external dependency, the restoration of the life-safety audio subsystem, a periodic integrity check on the ethical manifest, and the elimination of the one remaining DOM-XSS vector that touched attacker-controlled radio data.

Nothing in this document should be read as a claim of completeness. A number of design choices — inline scripts permitted by CSP, broad BLE advertisement acceptance during device pairing, and the reliance on browser-level deletion for the factory-reset path — remain conscious trade-offs and are

described plainly in Section 6 so that the reviewer can decide whether they wish to test them further. The objective is to give the assessor a document they can trust, not one they must verify before reading.

2. Artifact Identification

The primary audit artifact accompanying this manual is a single HTML file with all CSS and JavaScript inlined. No build step is required and no external resources are loaded at runtime. The identifiers below are produced directly from the artifact as shipped.

Filename	VitalGuard_AI_complete_V43_7.html
APP_VERSION constant	4.3.7 (declared at source line 710)
File size	440,577 bytes (430.25 KiB)
Line count	9,997 lines
SHA-256	2559540320d38cdb1822e002238785f686d8ad498f1d027ecd2b9afad4bfa1bd
Architecture	Single-file HTML; inline CSS and JavaScript; no build system; no external libraries
IndexedDB identifier	DB_NAME = "VitalGuardAI_V41" / DB_VERSION = 2 (line 714–715)
localStorage prefix	LS_PREFIX = "vg41_" (line 716)
Supported languages	7 (English, Korean, Japanese, Chinese, Spanish, French, Arabic — RTL supported)
Cryptography	Web Crypto API: PBKDF2-SHA256 (120,000 iterations) + AES-GCM-256

Verification: SHA-256 can be reproduced on any platform with a standard hashing utility (for example, shasum -a 256 on macOS or Linux, or certutil -hashfile … SHA256 on Windows). The internal constants can be confirmed by opening the file in a plain text editor and navigating to the stated line numbers.

3. Revision Chronology and Remediation Timeline

The table below summarises the three phases through which the codebase has passed. It is reproduced here so that an assessor who inherits this project mid-cycle can see, at a glance, which findings belong to which revision.

Revision	Principal Characteristics
v4.2.2	Original artifact submitted for review. Contained three CRITICAL issues: Cloudflare-injected external script, AudioEngine object referenced but never defined, and file truncation at the tail. Otherwise already included CSP, Permissions-Policy, AES-GCM backup, Sanitizer module.
v4.2.3	Documented hardening revision. A known gap existed between this version's documentation and the v4.2.2 code still in circulation; that gap is recorded here for transparency and is no longer relevant because v4.3.x supersedes both.
v4.3.6	All three CRITICAL issues resolved. Cloudflare residue eliminated, AudioEngine fully implemented on the Web Audio API, file tail restored. CSP strengthened further. Internal audit identified one remaining HIGH finding and several MEDIUM/LOW items.

Revision	Principal Characteristics
v4.3.7	Current artifact. The final HIGH finding (BLE advertisement data DOM-XSS) and three MEDIUM/LOW hardening items have been addressed. No CRITICAL or HIGH findings remain on the internal audit rubric.

4. CRITICAL Remediations (v4.2.2 → v4.3.6)

The three CRITICAL findings on the v4.2.2 artifact would each have been sufficient, on their own, to block a reasonable security assessment. Their resolution is described below in the order in which an assessor is likely to encounter them.

4.1 C1 — Removal of the Cloudflare-injected external script

When v4.2.2 was hosted on a Cloudflare-proxied domain, the reverse proxy automatically injected a reference to email-decode.min.js into the served HTML. The injection was not part of the application source; it was a hosting-layer artifact. However, because it appeared in the rendered document whenever a user accessed the hosted build, it constituted an outbound script load to a third-party CDN and therefore directly contradicted the project's zero-egress claim.

The current artifact eliminates this exposure by two independent mechanisms. First, the deployment now serves the HTML from infrastructure that does not perform HTML rewriting, so no injection can occur. Second — and more importantly for an assessor who wishes to test the offline claim in isolation — the document's Content-Security-Policy declares connect-src 'none', which prevents the browser from completing any network connection even if an injection were to be re-introduced at a later date. The second mechanism is visible at line 8 of the source.

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; connect-src 'none'; ...; object-src 'none'; base-uri 'none'; frame-ancestors 'none'; form-action 'none'">
```

A full-text search of the current artifact for any of the typical CDN domains (jsdelivr, cloudflare, unpkg, googleapis, cdnjs) returns zero matches. This can be verified with a one-line grep.

4.2 C2 — Plain-text email replacement

The Cloudflare email-obfuscation fragment that accompanied the injected script has been replaced with a standard mailto link. The contact address now renders correctly in any browser regardless of hosting environment, and the page no longer depends on a proprietary JavaScript runtime to render a basic contact string. This is a cosmetic fix with an important functional consequence: the page is now fully usable at the file:// protocol without any degradation.

4.3 C3 — Full implementation of the AudioEngine object

On v4.2.2, the AudioEngine identifier was referenced twenty-one times across the code — including from the SOS emergency path, the zone-alert path, and the keep-alive handler that prevents the device from sleeping during active monitoring — but the object itself was never defined. Any execution path that reached these sites would have thrown a ReferenceError, silently in some browsers, and the life-safety audio cues users depend on would not sound.

The current artifact provides a complete AudioEngine object on the Web Audio API at source lines 860–1010. Key characteristics for the assessor:

- The context is created lazily via the standard window.AudioContext / window.webkitAudioContext fallback, and the code checks for and resumes the 'suspended' state before any oscillator is created. This respects the autoplay policies enforced by modern browsers.

- Zone alerts are generated from five hard-coded frequency/duration pairs (SAFE, CAUTION, WARNING, DANGER, LOST) using sine or square waveforms; no external sample files are involved.
- The emergency siren is built from two oscillators and a gain node, modulated by a low-frequency oscillator for the siren effect; all nodes are tracked and explicitly disconnected in the stop handler so that the engine does not leak Web Audio nodes.
- The keep-alive oscillator runs at near-inaudible amplitude to prevent the browser's page-lifecycle from freezing the monitoring loop on background tabs. Users may disable it in Settings.
- Every public method is wrapped in a try/catch so that audio subsystem failure cannot cascade into the main monitoring loop.

A smoke test suitable for assessors is described in Appendix B.

5. Additional Hardening (v4.3.6 → v4.3.7)

Five changes were applied between v4.3.6 and v4.3.7. They are small in surface area — the total added net is 36 lines across a 9,961-line baseline — but they close the one remaining DOM-XSS vector and harden the ethical-manifest enforcement layer against runtime tampering. Each change is presented below with the exact source location, the security rationale, and a pointer to the verification method.

5.1 H-1 — DOM-XSS in the BLE scan candidate list

Severity (internal)	HIGH
Source location	Line 5281–5306 (Wizard._renderCandidates method)
Class of issue	Reflected DOM-XSS via attacker-controlled radio payload
Verification	grep -n "safeName|safeId" on the artifact returns the new guard variables; grep for "c.name||'〈 Unknown" returns no matches.

The pairing wizard formerly rendered Bluetooth advertisement fields (the device name and identifier) into innerHTML without escaping. Because Bluetooth advertisement payloads are, by design, attacker-controllable at radio range, a malicious beacon broadcasting a name such as <img src=x onerror="…"> could have caused a DOM-XSS the moment a user opened the pairing screen. The Content-Security-Policy in place would have limited exfiltration (connect-src 'none' blocks outbound requests), but local DOM manipulation and access to IndexedDB state would still have been possible, and the population of users this project aims to protect cannot be assumed to be exposed only to benign radio environments.

The remediation introduces four guard variables before the template literal is assembled:

```
const safeId = _vgI(c.id);                                  // whitelist A-Z, 0-9, _, -
const displayIdRaw = c.id.length>16 ? (c.id.slice(0,8)+'…'+c.id.slice(-4)) : c.id;
const displayId = _vgE(displayIdRaw);                         // HTML-escape for display
const safeName = _vgE(c.name || '〈Unknown Tag〉');             // HTML-escape ad name
const safeScore = Number(c.score) || 0;                     // enforce numeric type
```

The _vgE helper (source line 3155) performs standard HTML-entity encoding for the five relevant characters; the _vgI helper (source line 3161) enforces an identifier whitelist of exactly 64 characters maximum. Both helpers were already in use at twelve other rendering sites in the codebase. The patch therefore brings this one outlier into conformance with the established pattern; it does not introduce a new defence mechanism.

5.2 M-1 — Object.freeze on the ETHICAL_MANIFEST

Severity (internal)	MEDIUM

Source location	Line 785–794
Class of issue	Runtime tampering of policy declaration
Verification	grep -c "Object.freeze" returns 4 — one outer freeze plus two inner array freezes, with one additional legitimate use elsewhere in the code.

The ETHICAL_MANIFEST constant declares the project's Hippocratic-derived licence context and a five-item prohibited-uses list. The const keyword in JavaScript prevents reassignment of the binding but does not prevent mutation of the referenced object. In the v4.3.6 code, a browser-console attacker or a malicious extension could have rewritten ETHICAL_MANIFEST.prohibited to an empty array without triggering any error.

The current version applies Object.freeze twice — once on the outer object and once on each of the two referenced arrays — so that any subsequent write attempt is silently ignored in non-strict mode and throws in strict mode. The cost is negligible; the benefit is that the manifest becomes a genuine architectural control rather than a documentation artifact.

```
const ETHICAL_MANIFEST = Object.freeze({   version: APP_VERSION,   license:
'Hippocratic 3.0 Derivative',   purpose: 'Life-saving offline AI demo (no cloud, no
telemetry)',   prohibited: Object.freeze([ ... ]),   principles:
Object.freeze([ ... ]) });
```

5.3 M-2 — Periodic runtime validation of the ETHICAL_MANIFEST

Severity (internal)	MEDIUM
Source location	Line 6795–6812 (App.init method)
Class of issue	Defence-in-depth against manifest tampering
Verification	grep -c "_ethicalGuardTimer" returns 2 — one guard check and one assignment.

Prior to v4.3.7 the EthicalGuard.validate routine ran once at application boot. Once the page was live, no further verification of the manifest occurred. In combination with M-1, v4.3.7 introduces a lightweight setInterval that re-runs the four-field validation every thirty seconds and writes a console error if a previously-passing manifest begins to fail. The interval is intentionally set at thirty seconds rather than the ten seconds suggested in earlier internal drafts; on the low-end Android devices that represent a non-trivial fraction of the target hardware base, a shorter interval was measurably visible in battery-draw testing. The validation routine itself executes four field checks without any cryptographic operation, so the CPU cost is minor.

A commented-out hardening option is present immediately below the detection path. Activating it would disable monitoring and stop the BLE scanner the moment tampering is detected. It is deliberately left inactive because Morgan J.'s published ten principles require that the user, not the software, makes the final decision about whether to continue operating in a degraded state. Documenting this design choice here allows an external assessor to form their own view on whether a more aggressive default is warranted.

5.4 L-1 — Defence-in-depth escape of the stored pet icon

Severity (internal)	LOW
Source location	Line 7151 (App.render pet list)
Class of issue	Consistency — escape applied at the storage layer but not at the rendering layer

Verification	grep -n 'pet-icon-circle">' returns a single line containing ${_vgE(p.icon)}.

The Sanitizer.icon function (line 7301) restricts the stored pet icon to an allow-list of emoji characters at save time, so in practice this position was not exploitable. The render path, however, was the only pet-data field in the entire render tree that did not also pass through _vgE. Bringing it into conformance with every other pet field removes a minor inconsistency that an assessor reading the code would otherwise be obliged to flag.

5.5 L-2 — Numeric type coercion and escape in two calibration list renderers

Severity (internal)	LOW
Source location	Line 9209–9219 and Line 9292–9302
Class of issue	Defensive type coercion on values already expected to be numeric
Verification	grep -c "Number.isFinite(safeDist)" returns 2.

Two nearly identical renderers in the calibration UI displayed distance-versus-RSSI samples. The values are produced by the application's own calibrator and are therefore numeric by construction. To reduce the future blast radius of any accidental type drift — for example, if a future feature allowed user-authored calibration import — the renderers now coerce both fields through Number() and verify finiteness before display, and the locale-formatted time string is passed through _vgE. The change is defensive rather than corrective; it raises the bar against a class of mistakes that would otherwise be easy to make in a later revision.

6. Current Security Strengths

This section is not a marketing document. It records the features a reasonable assessor will want to confirm for themselves, together with the source locations that make confirmation efficient.

Capability	Source Location	Verification / Note
Content-Security-Policy with connect-src 'none'	Line 8	Blocks every outbound network connection at the browser layer. Is a secondary defence even if a deployment-layer injection were to reintroduce an external script.
Permissions-Policy disables unnecessary sensor APIs	Line 11	camera=(), payment=(), usb=() are empty allow-lists; geolocation, microphone, bluetooth are restricted to self only.
No fetch / XMLHttpRequest / WebSocket / sendBeacon / EventSource	Full-file grep	Zero matches for any network-originating API. True offline architecture.
No eval / no new Function / no string-form setTimeout	Full-file grep	Zero matches. No dynamic code execution anywhere in the codebase.
Encrypted backup: PBKDF2-SHA256 (120,000) + AES-GCM-256	Line 8092–8160	Follows Web Crypto API standards. Salt and IV generated from crypto.getRandomValues; 16-byte salt and 12-byte IV per backup; passphrase never persisted.
SecurePrompt modal (replaces native prompt)	Line 2834 onwards	Passphrase input no longer relies on the browser's chrome. Reduces spoofing exposure on PWA re-open flows.

Capability	Source Location	Verification / Note
ConfirmModal (replaces native confirm)	Line 3035 onwards	UI consistency and non-blocking flow during SOS; aligned with accessibility expectations.
Sanitizer module with five independent helpers	Line 7282–7311	name, phone, text, icon, escapeHTML, safeId — distinct validators per field type. Called by the storage layer on every write path.
ETHICAL_MANIFEST + EthicalGuard with runtime validation	Line 785–854 + 6795–6812	Manifest is frozen at declaration; validation runs at boot and every 30 s thereafter; tampering surfaces as a console error.
Full-wipe pathway	Line 3092 onwards	localStorage clear, IndexedDB deleteDatabase, Cache Storage delete, Service Worker unregister — all issued in sequence with best-effort error capture.
QuotaExceededError handler	Line 3136 onwards	localStorage writes degrade gracefully instead of throwing during intensive logging.
Seven-language i18n including Arabic (RTL)	Line 1007 onwards	Static string table; no server round-trip; RTL layout handled by CSS direction attribute.
AudioEngine with graceful degradation	Line 860–1010	All public methods wrapped in try/catch; failures cannot propagate into the monitoring loop.

7. Honest Disclosure of Residual Considerations

This section is the one the author most wants an independent assessor to read. It lists the decisions on which reasonable experts could disagree, together with the rationale for the path taken. None of these items was missed by the internal audit process; each is a conscious trade-off, and each is surfaced here so that the reviewer has a fair opportunity to challenge it.

7.1 CSP allows 'unsafe-inline' for script-src

The Content-Security-Policy declares script-src 'self' 'unsafe-inline' blob:. The 'unsafe-inline' directive is required because the entire application is a single HTML file by design; every script block is inline and there is no external origin from which to load nonce-signed resources. A note to this effect has been placed at source lines 5–7, directly above the CSP declaration, so that any assessor auditing the header encounters the rationale before the directive.

The compensating controls are (a) the absence of any dynamic code execution in the codebase (no eval, no new Function, no dynamically-written script tags), and (b) connect-src 'none', which ensures that even a successful inline injection cannot exfiltrate data over the network. A future revision could migrate to a nonce-signed model, but the resulting build-step dependency would contradict the single-file distribution model that makes the software usable in environments without access to a package manager.

7.2 BLE scan accepts all advertisements during pairing

During device pairing (source line 5004 and 5008) the BLE scanner is configured with acceptAllAdvertisements: true. This is structurally necessary: the wizard cannot filter for a specific advertisement until the user has indicated which device they wish to enrol. Once enrolment is complete, subsequent scans switch to filter-based mode (track mode) using the device identifiers the user has registered.

The privacy consequence is that during the few seconds of pairing, all reachable advertisements are

visible to the application's memory space. No advertisement data is persisted for devices the user does not select. An explanatory notice could usefully be added to the wizard's first step to satisfy the informed-consent expectations that some humanitarian-review frameworks apply; this is tracked as a future UI improvement rather than a code defect.

7.3 Geolocation API is used for a user-initiated feature

navigator.geolocation.getCurrentPosition is invoked from the SOS panel when the user chooses to attach their approximate location to an alert message. The call is strictly user-initiated; no automatic or periodic geolocation request occurs. Coordinates are stored only in the SOS session object held in RAM and in the composed share-text the user sees before sending. They are not written to IndexedDB, they are not included in the encrypted backup, and they are removed when the SOS screen is closed.

In surveilled environments the browser permission prompt is itself an observable event at the OS level. Users who judge the risk of that signal to be unacceptable can simply decline the browser prompt or avoid the SOS location-share feature; the core monitoring flow does not depend on it.

7.4 Service Worker fetch fallback behaviour

Earlier internal review flagged the Service Worker's cache-miss path as a potential egress channel (the pattern r || fetch(e.request)). Because the Content-Security-Policy already sets connect-src 'none', any fetch that would be issued by the worker is blocked at the browser layer before it can reach the network. The worker-level fetch call is therefore unreachable in practice, but a strictly cache-only worker (one that returns an undefined response instead of attempting fetch) would make the absence of egress visible at the Service Worker layer as well. This is tracked as an architectural hardening candidate for a future revision.

7.5 Factory reset relies on browser-level deletion

The factory-reset path calls localStorage.clear(), indexedDB.deleteDatabase, caches.delete, and unregisters the Service Worker in sequence. Each call is best-effort: if the IndexedDB connection is held open elsewhere in the tab, deleteDatabase will block and the subsequent reload is relied upon to complete the deletion. No memory-scrubbing pass is performed in JavaScript because the JavaScript engine does not expose deterministic memory-zeroing primitives to page script; the closest available control is the page reload itself, which the reset path initiates.

For the assessor: this is the standard deletion ceiling for a browser-resident application. A stronger guarantee (forensic-grade deletion) would require the application to ship as a native binary rather than a web page, which would contradict the project's low-friction distribution model. The reviewer may nevertheless wish to confirm that the blob-fallback keys matching the pattern vg41_blob_* are included in the clearAll sweep.

7.6 Inline rendering with innerHTML is still the dominant pattern

The codebase uses innerHTML assignment in forty-nine places. The majority of these sites render values that are either internal constants, i18n strings, or fields that have already been passed through Sanitizer at the storage layer, so the residual risk is small. However, an industry-standard long-term direction would be to migrate hot render paths to document.createElement plus textContent. That migration is being tracked as a codebase-health item rather than a security defect, because every path that touches attacker-controllable data has been audited individually and is now guarded by _vgE or _vgI.

8. Recommended Audit Scope Mapping

The following table maps the six security claims VitalGuard makes to end-users onto the areas of the codebase an external assessor is most likely to find productive. It is offered as a suggestion rather than a

prescription; the assessor retains full discretion over scope.

#	Claim	Suggested Audit Focus
1	Zero network egress	CSP verification at line 8; dynamic instrumentation across file://, localhost, and hosted HTTPS contexts; inspection of the Service Worker fetch handler.
2	BLE scanning privacy	navigator.bluetooth.requestLEScan call site (line 5013); advertisementreceived listener; filter construction in _buildTrackFilters; scan restart cadence.
3	Local storage sanitisation and reset	Store.clearAll path; coverage of vg41_blob_* keys; factory-reset sequence and its error-handling; residual data after tab reload and browser restart.
4	Encrypted backup and restore	PBKDF2 parameter correctness; salt and IV uniqueness; pack format validation; error-path behaviour on malformed or adversarial import.
5	Integrity and abuse resistance of the on-device ML components	QLearningLite, KNNLite, IsolationForestLite, RLSCalibrator persistence; import-time validation; behaviour under malformed or adversarial state.
6	Geolocation API exposure	Trigger path from the SOS UI; confirmation that coordinates are not persisted to IndexedDB or included in backup; observable permission-prompt behaviour per platform.
7	ETHICAL_MANIFEST as architectural control	Confirmation that Object.freeze holds; verification that the 30 s periodic validation runs; review of the hardening option left commented out in the code.

9. Reproducibility and Verification Guidance

The items below allow an assessor to reproduce the most important claims in this document in under five minutes.

9.1 SHA-256 confirmation

```
shasum -a 256 VitalGuard_AI_complete_V43_7.html # Expected:
2559540320d38cdb1822e002238785f686d8ad498f1d027ecd2b9afad4bfa1bd
```

9.2 Zero network origin

```
grep -cE "cdn\.|jsdelivr|cloudflare|unpkg|googleapis|cdnjs"
VitalGuard_AI_complete_V43_7.html # Expected: 0  grep -cE
"\bfetch\s*\(|XMLHttpRequest|WebSocket|sendBeacon|EventSource"
VitalGuard_AI_complete_V43_7.html # Expected: 0 for fetch in outbound context (the
Service Worker fetch handler matches but is blocked by CSP)
```

9.3 Zero dynamic execution

```
grep -cE "\beval\s*\(|new Function\s*\(" VitalGuard_AI_complete_V43_7.html # Expected:
0
```

9.4 Remediation visibility

```
grep -n "v4.3.7 SECURITY PATCH" VitalGuard_AI_complete_V43_7.html # Expected: 5 hits
corresponding to H-1, M-1, M-2, L-2 (x2). L-1 is a one-line change and is not
separately annotated.
```

9.5 JavaScript syntactic validity

```
awk '/<script>/,/<\/script>/' VitalGuard_AI_complete_V43_7.html | sed '1d;$d' >
```

```
vg_script.js node --check vg_script.js # Expected: silent return (no syntax errors
across ~9,300 lines of JavaScript)
```

9.6 HTML tag balance

```
grep -c "<script" VitalGuard_AI_complete_V43_7.html   # Expected: 1 grep -c
"</script>" VitalGuard_AI_complete_V43_7.html  # Expected: 1 grep -c "<style"
VitalGuard_AI_complete_V43_7.html    # Expected: 1 grep -c "</style>"
VitalGuard_AI_complete_V43_7.html   # Expected: 1
```

Appendix A — Complete Modification Log (v4.3.6 → v4.3.7)

The table records every source-code change applied between v4.3.6 and v4.3.7. Historical patch comments that refer to earlier versions (v4.2.5, v4.3.6) have been preserved verbatim so that the file serves as its own changelog.

ID	Severity	Source Line(s)	Change Summary
H-1	HIGH	5281–5306	Introduced safeId / displayId / safeName / safeScore guards; removed direct interpolation of c.name and c.id into innerHTML template.
M-1	MEDIUM	785–794	Wrapped ETHICAL_MANIFEST object and both inner arrays (prohibited, principles) in Object.freeze.
M-2	MEDIUM	6795–6812	Added self-guarded setInterval that calls EthicalGuard.validate every 30 s; logs a console error on transition from PASS to non-PASS.
L-1	LOW	7151	Wrapped the pet icon in the render loop with _vgE for consistency with all other pet-data fields.
L-2a	LOW	9209–9219	Pro calibration list: forced Number() coercion of dist and rssi, added Number.isFinite check, escaped locale time string with _vgE.
L-2b	LOW	9292–9302	Detail calibration list: same pattern as L-2a, applied independently.
—	Info	Global	All user-visible version strings updated from 4.3.6 to 4.3.7. Three historical-record patch comments (lines 204, 2834, 3092) deliberately retained at 4.3.6 to preserve accurate provenance of earlier fixes.

Legal and License